

## 5교시: Docker 기본 명령어 1 - 실행 80% 사이클

### 수업 목표

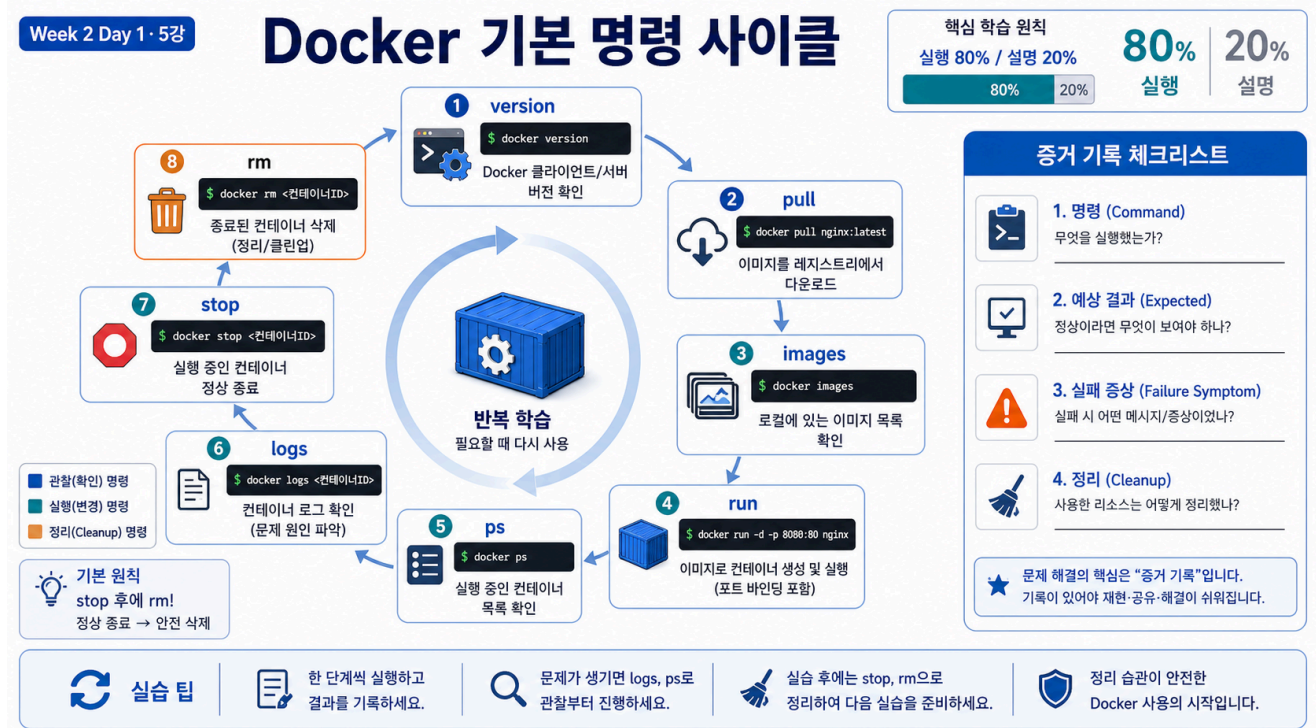
- Docker 기본 명령어를 암기 목록이 아니라 상태 확인과 lifecycle 제어 도구로 사용한다.
- version, pull, images, run, ps, logs, stop, rm을 실행 목적과 evidence 기준으로 연결한다.
- 실행 후 중지과 정리까지 포함한 하나의 hands-on cycle을 완성한다.

### 50분 흐름

| 시간     | 활동                                            | 비중     | 학생 산출                   |
|--------|-----------------------------------------------|--------|-------------------------|
| 0-5분   | 실습 목표와 안전 기준 확인                               | 설명 10% | command 목적 note         |
| 5-15분  | docker version, docker pull, docker images 실행 | 실행 20% | image 준비 evidence       |
| 15-30분 | docker run, docker ps로 container 상태 확인        | 실행 30% | running status evidence |
| 30-40분 | docker logs, docker stop, docker rm 실행        | 실행 20% | log/cleanup evidence    |
| 40-47분 | Linux 테스트 결과와 자기 출력 비교                        | 실행 10% | 차이 기록                   |
| 47-50분 | 정리와 다음 교시 연결                                  | 설명 10% | 질문 1개                   |

실행 비중은 약 80%다. 설명은 명령의 목적과 실패 증상만 잡고, 나머지 시간은 학생이 직접 실행하고 결과를 기록하는 데 사용한다.

## Visual 1: Docker 기본 명령 사이클



이 이미지는 version -> pull -> images -> run -> ps -> logs -> stop -> rm 흐름을 한 사이클로 보여준다. run에서 끝내지 않고 관찰과 정리까지 완료해야 하나의 실습이 끝난다.

## 실습 전 기준

오늘 명령은 모두 로컬 Docker와 Docker Hub public image를 사용한다. cloud resource는 만들지 않는다. container 이름은 충돌을 피하기 위해 paperclip-day1-nginx를 사용한다. 기존에 같은 이름의 container가 있으면 먼저 정리해야 한다.

```
docker ps -a --filter name=paperclip-day1-nginx
```

출력이 비어 있으면 바로 진행한다. 같은 이름의 container가 보이면 6교시 정리 절차에 따라 stop과 rm을 수행한다.

## 기본 명령 목적표

| 명령                       | 실행 목적                   | 정상 evidence             | 흔한 실패                      |
|--------------------------|-------------------------|-------------------------|----------------------------|
| docker version           | CLI와 daemon 연결 확인       | Client/Server 정보가 보임    | daemon 미실행, 권한 문제          |
| docker pull nginx:latest | registry에서 image 확보     | digest와 status 출력       | network/auth/image name 문제 |
| docker images            | local image 목록 확인       | nginx, hello-world 등 표시 | socket 권한 문제               |
| docker run               | image에서 container 시작    | container ID 또는 output  | port 충돌, image 없음          |
| docker ps                | running container 확인    | STATUS, PORTS, NAMES 확인 | 필터/이름 오타                   |
| docker logs              | process log 확인          | entrypoint/start log    | container 이름 오타            |
| docker stop              | running container 정상 중지 | container 이름 반환         | 이미 종료됨                     |
| docker rm                | stopped container 삭제    | container 이름 반환         | 실행 중이라 삭제 실패               |

## Hands-on 1: Docker 연결과 image 준비

```
docker version
docker pull nginx:latest
docker images
```

### Linux 사전 테스트 결과

환경: Ubuntu 24.04.3 LTS, Linux 6.6.87.2-microsoft-standard-WSL2, Docker Client 29.0.2, Docker Server 29.3.1.

docker version 핵심 출력:

```
Client: Docker Engine - Community
 Version:           29.0.2
 OS/Arch:           linux/amd64

Server:
 Engine:
  Version:          29.3.1
 OS/Arch:           linux/amd64
```

docker pull nginx:latest 재실행 시 출력:

```
latest: Pulling from library/nginx
Digest:
sha256:5aca99593157f4ae539a5dec1092a0ad8762f8e2eb1789085a13a0f5622369f6
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

처음 실행하는 학생은 Downloaded newer image가 보일 수 있다. 이미 받은 학생은 Image is up to date가 보일 수 있다. 둘 다 정상이다.

## Hands-on 2: container 시작과 상태 확인

```
docker run -d --name paperclip-day1-nginx -p 18080:80 nginx:latest
docker ps --filter name=paperclip-day1-nginx
```

### 기대 결과

docker run -d는 긴 container ID를 출력한다. docker ps는 다음과 비슷하게 보인다.

| CONTAINER ID | IMAGE        | COMMAND                  | STATUS        | PORTS                                     |
|--------------|--------------|--------------------------|---------------|-------------------------------------------|
| ce1d01f8a780 | nginx:latest | "/docker-entrypoint...." | Up 44 seconds | 0.0.0.0:18080->80/tcp, [::]:18080->80/tcp |
|              |              |                          |               | paperclip-day1-nginx                      |

PORTS에서 18080->80/tcp가 핵심이다. host의 18080 port로 들어온 요청이 container 내부의 80 port로 전달된다.

## Hands-on 3: log 확인과 정리

```
docker logs paperclip-day1-nginx
docker stop paperclip-day1-nginx
docker rm paperclip-day1-nginx
docker ps --filter name=paperclip-day1-nginx
```

### Linux 사전 테스트 결과

docker logs 핵심 출력:

```
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/06/04 04:07:14 [notice] 1#1: nginx/1.31.1
```

```
2026/06/04 04:07:14 [notice] 1#1: start worker processes
```

docker stop과 docker rm 출력:

```
paperclip-day1-nginx
paperclip-day1-nginx
```

정리 후 docker ps --filter name=paperclip-day1-nginx 출력:

| CONTAINER ID | IMAGE | COMMAND | CREATED | STATUS | PORTS | NAMES |
|--------------|-------|---------|---------|--------|-------|-------|
|--------------|-------|---------|---------|--------|-------|-------|

헤더만 보이면 running container가 남아 있지 않은 상태다.

## Evidence 기록 양식

| 항목                              | 학생 기록                |
|---------------------------------|----------------------|
| docker version Client/Server 확인 |                      |
| nginx:latest pull 결과            |                      |
| container name                  | paperclip-day1-nginx |
| port binding                    | 18080:80             |
| docker ps STATUS/PORTS          |                      |
| docker logs 핵심 1줄               |                      |
| stop/rm 완료 여부                   |                      |
| screenshot filename             |                      |

## 흔한 오해

| 오해                              | 바로잡기                                                       |
|---------------------------------|------------------------------------------------------------|
| docker run만 성공하면 끝이다.           | ps, logs, stop, rm까지 해야 운영 cycle이 닫힌다.                     |
| docker images에 기존 이미지가 많으면 문제다. | 기존 실습 이력이 있을 수 있다. 오늘 image인 nginx, hello-world를 구분해 기록한다. |
| Image is up to date는 실패다.       | 이미 최신 image가 local에 있다는 뜻이다.                               |
| rm을 먼저 하면 된다.                   | running container는 보통 먼저 stop한 뒤 rm한다.                     |

## 평가 기준

| 기준    | 2점 evidence                                     |
|-------|-------------------------------------------------|
| 실행 비중 | 직접 명령을 실행하고 결과를 기록했다.                           |
| 상태 확인 | version, images, ps, logs 중 3개 이상의 출력 의미를 설명했다. |
| 정리    | stop과 rm 후 남은 container가 없음을 확인했다.              |
| 재현성   | container name, port, image tag를 정확히 기록했다.      |
| 보안/비용 | cloud resource를 만들지 않고 local 실습만 수행했다.          |

## 공식 근거 링크

- Docker Docs: Docker run reference, <https://docs.docker.com/reference/cli/docker/container/run/>
- Docker Docs: docker container ls, <https://docs.docker.com/reference/cli/docker/container/ls/>
- Docker Docs: docker container logs, <https://docs.docker.com/reference/cli/docker/container/logs/>

## 다음 연결

다음 교시는 같은 명령을 hello-world와 nginx 서비스 확인 흐름으로 확장한다. 핵심은 "container가 떠 있다"가 아니라 HTTP 응답과 log로 서비스 상태를 검증하는 것이다.